Bachelor's Thesis

Implementing Floating-Point Support in a Nanopass-Python Compiler

Marius Spill

Examiner: Prof. Dr. Peter Thiemann

Albert-Ludwigs-University Freiburg

Faculty of Engineering

Department of Computer Science

Chair for Programming Languages

September 1, 2026

Writing period

June 26, 2026 – September 28, 2026

Examiner

Prof. Dr. Peter Thiemann

Declaration

I hereby declare, that I am the sole author and composer of my thesis and that no other sources or learning aids, other than those listed, have been used. Furthermore, I declare that I have acknowledged the work of others by providing detailed references of said work.
I also hereby declare that my thesis has not been prepared for another examination or assignment, either in its entirety or excerpts thereof.

The use of Large Language Models was limited to refining formulations rather than providing original input, assisting in researching concepts and sources, and tracing inaccuracies and bugs. Claude Sonnet 5 was used for more technical questions, while ChatGPT 5.5 and 5.6 were used for sources and formulations.

Place, Date	Signature

Abstract

In this thesis, I extended a Nanopass Python-to-RISC-V compiler to support floating-point values and their respective operations.

Furthermore, I extended the interpreter that defines the language's semantics and created a test suite of 23 test files to validate the compiler's behavior against it.

Finally, I discuss various design alternatives and justify the choices I made by weighing them against two guiding principles: staying as close to Python's existing semantics as possible, and preserving as much of the existing compiler architecture as possible.

This thesis describes the process, the decisions, and the resulting artifacts.

Contents

1 Introduction

The Nanopass architecture features a series of independent passes, each of which performs one small conversion or task, bringing the intermediate representation (IR) closer to the final assembly output.

The strength of this architecture is that it is highly extensible [Siek, 2023], allowing the existing compiler to be extended without overhauling the complete architecture.

The textbook version and the lecture stop at the implementation of integers, booleans, and tuples, without any support for floating-point values. Yet any sufficiently sophisticated program will eventually need to work with floating-point values, since they are an integral part of any programming language.

While the Nanopass architecture itself is highly extensible, the implementation still came with some hurdles, such as stack and heap allocation, register selection, instruction selection, and keeping track of type information to select the correct behavior at certain points in the pipeline (e.g., when calling a print function or selecting the appropriate register).

Another challenge was that, unlike ints, bools, and other data types, no bit could be reserved for tagging inside a float's own word, since the encoding requires all 64 bits for the value itself.

My implementation offers 64-bit floating-point support via IEEE 754 encoding, along with coercion for all original operations. These floating-point values use designated floating-point instructions from the RISC-V

architecture.

To avoid modifying the existing architecture, the extension was built alongside existing principles (such as bit tagging) rather than overhauling them. For situations where integers or pointers were tagged at the LSB, floats were instead boxed, with an encoding attached in the header.

The garbage collector was extended to check each object's header before following its fields as pointers, skipping the payload entirely for boxed floats, since their fields never contain pointers.

Furthermore, I extended the interpreter that defines the language's semantics and created a test suite of 23 tests to validate the compiler's behavior against it.

Finally, in this thesis, I discuss design decisions and the pros and cons of various approaches, and document the implementation and its results.

2 Background

2.1 The Nanopass Compiler

The compiler's architecture is based on Jeremy Siek's textbook *Essentials of Compilation* [Siek, 2023]. The main idea is that instead of performing one huge transformation, the compilation process is separated into a sequence of passes, each with its own target representation and task.

The first step is parsing the input program into an AST. Several third-party tools and libraries exist for parsing, and building one is not the scope of this thesis; this compiler relies on Python's own `ast` module [Python Software Foundation, 2026a]. Once a Python AST is available, each pass has a designated and well-specified task, leading to a clean architecture in which each module has its own job [Martin, 2008].

Among these passes are, for example, flattening out functions, bringing a nested expression into a list of atomic expressions (monadic normal form), selecting instructions, and allocating addresses and space on the stack or heap.

While the textbook and Professor Thiemann's lecture cover many concepts of a compiler, support for floating-point values and their operations was not present in the textbook or lecture version of the compiler. During this thesis, I explored how the existing architecture could be extended to support floating-point values, which design decisions had to be made, and what alternatives existed.

I ultimately arrived at an implementation of one such approach, which is discussed in the following chapters.

2.2 RISC-V

The target architecture of the compiler is the RISC-V instruction set architecture [Waterman et al., 2016]. RISC-V is an open instruction set architecture, originally developed at the University of California, Berkeley.

RISC-V's design philosophy favors a reduced number of supported instructions, making it a comparatively simple and abstract architecture to target from a compiler.

Unlike proprietary architectures, RISC-V is maintained as an open standard rather than by a single vendor, with multiple independent hardware and software implementations [Waterman et al., 2016].

2.3 IEEE 754 Floating Point Representation

For encoding floating-point values, I decided to follow the industry standard, IEEE 754 [IEEE, 2019]. Under this standard, a double-precision float is represented as a 64-bit word consisting of 1 sign bit, an 11-bit exponent, and a 52-bit mantissa.

Double precision is natively supported by the RISC-V architecture and matches the precision Python uses internally to represent floats [Python Software Foundation, 2026b]. A problem emerged with the existing compiler design, in which the least significant bit (LSB) of every value and pointer was used for tagging - something that is not possible with this encoding, since it requires all 64 bits.

This was solved by boxing floats and encoding their type in the header of the box (see chapters 3 and 4).

3 Design Architecture

3.1 Design Philosophy

The governing principle of this extension was functionality first, optimisation second. This drove two central decisions. First, floats are represented as 64-bit doubles rather than 32-bit single-precision values, in order to preserve fidelity with Python's native semantics. Second, floats are stored as boxed heap objects rather than as raw values, accepting the cost of indirection in exchange for reusing the compiler's existing tuple and garbage-collection machinery. Performance optimisations were partly explored as part of this work, with the remainder left for future work.

A related decision concerns mixed-type arithmetic. In any operation mixing an integer and a float, the integer operand is implicitly promoted to a float before the operation is performed, matching Python's native behaviour. This keeps a single arithmetic path for floats and confines the type difference to a conversion step at the operands. The design and placement of this conversion are discussed in Section 3.3.

3.2 Float Representation on the Heap

The garbage collector uses bit tagging to distinguish pointers, which it must follow, from raw values, which it must leave untouched. Integers are stored shifted one bit to the left, so that their least-significant bit (LSB) is 0, marking

the word as a value the collector should not trace. Heap pointers instead carry an LSB of 1, signalling the collector to follow them. Every structure stored on the heap (tuples, closures, and so on) is laid out as a block of words prefixed by a header word.

A double in the IEEE-754 format cannot be tagged in this way, because all 64 bits are needed to encode the value, leaving no bit free for a tag. Worse, the bit that serves as the tag for integers corresponds, in a double, to part of the mantissa; repurposing it would distort the value entirely.

Floats must nonetheless be storable on the heap, since the language permits them as fields of objects, as function arguments, and as elements of tuples. Restricting the language to avoid this was rejected on principle, as the guiding philosophy stated above places functionality over performance. A representation compatible with the existing tagged collector was therefore required.

3.2.1 Alternatives

One option would have been to store floats as raw values directly on the heap. This would require overhauling the garbage collector to move away from bit tagging towards a layout map, that is, a description of the heap that records, for each structure, which words are pointers and which are raw values. Such a scheme would offer a scalable and memory-efficient way to represent any value directly on the heap, and would allow values to be stored inside any other structure without indirection. The drawback is that it conflicts with the architecture of the compiler as built so far, and that constructing and maintaining layout maps would introduce considerable overhead.

The approach I chose instead was to box every float inside a two-word heap block: one header word and one word holding the value of the float. The header identifies the block as a boxed float. This yields a uniform representation that can be used for literals, for variables, and for floats nested

in any heap structure, all through the same interface.

The intended behaviour is for the collector to recognise a boxed float from its header and to copy the payload word verbatim, without interpreting those raw bits as a pointer. Section 3.4 describes how this header-aware handling was implemented.

The advantage of boxing is that it offers a uniform design that delivers full functionality with little overhead and fits almost seamlessly into the existing compiler. The disadvantage is the inefficiency that arises from having to reach inside the box on every access to the value, and from allocating a fresh box for every float that is produced, which increases pressure on the collector. These costs were judged acceptable, since the trade-off aligned with the design philosophy of providing functionality before performance.

3.3 Conversion of Integers to Floats in Coercion Expressions

Python implicitly promotes integers to floats in mixed-type operations—`3 + 3.14` evaluates to `6.14`, not a type error. Supporting this behaviour follows directly from the design philosophy of preserving Python's native semantics.

The detection of cross-type operations is performed in the heap allocation pass. This pass already identifies floats to box them, and more generally decides what must happen to each piece of data—whether it needs boxing, unboxing, or conversion. Placing the coercion logic here keeps these semantic decisions in one place.

When the pass detects an operation in which at least one operand is a float and the other is an integer, it wraps the integer operand in a new unary operation, `int_to_float`. Structurally, this operation behaves just like any other unary expression (such as negation); it flows through the subsequent passes unchanged until it reaches instruction selection, where it is lowered to

the RISC-V `fcvt.d.l` instruction that converts a 64-bit integer value to a double-precision float.

3.3.1 Alternatives

One alternative would have been to introduce a dedicated coercion pass before the heap allocation pass. This would separate concerns more cleanly, since one could argue that the purpose of the allocation pass is heap allocation, not type coercion. While technically correct, the trade-off would be introducing an entire new pass for a single transformation in one specific case. This option was rejected as disproportionate; however, should the language require additional type conversions in the future, a unified coercion pass would become architecturally justified.

A second and simpler alternative would have been to handle coercion entirely in instruction selection, using the type dictionary to detect mixed-type operands and emitting the conversion inline. This would confine the change to a single file, making it the least complex solution. The drawback is that instruction selection would take on semantic work—deciding *what* conversion is needed—on top of its existing role of deciding *how* to lower operations to machine instructions. The chosen approach keeps these responsibilities separate: the allocation pass decides that a conversion is needed, and instruction selection decides how to implement it. This separation also makes a future migration to a dedicated coercion pass straightforward, since the conversion is already represented as an explicit operation in the intermediate representation.

3.3.2 Possible Improvements

Efficiency could be improved by replacing bit tagging with a full layout or type map, which would permit storing raw floats directly on the heap rather than boxing them. Complementarily, the register allocator could be extended

to distinguish floats from integers and to maintain a separate region of the stack for floating-point values, so that raw floats could be spilled to the stack just like integers, instead of being boxed and sent to the heap.

3.4 Garbage Collector Extension

Since bit tagging doesn't work for floats - because they need the whole 64 bits - they are boxed. Yet once the Garbage Collector follows the pointer to a boxed float, it still has to decide whether the box's payload is itself a pointer it needs to follow, or just a raw value. To solve this, the header of a boxed float gets a second tag bit (bit 1) marking it as raw data.

Simple as that - if the Garbage Collector reads a header, it checks bit 1, and if it is set, skips over the payload instead of scanning it for pointers, then continues as usual.

The simplicity and seamlessness of this approach is why I decided for it, but there is a more structural reason as well. The collector is a Cheney-style copying collector [Cheney, 1970], meaning it scans to-space sequentially rather than by following individual pointers. By the time the scan reaches a given object, it no longer knows which pointer led to it - the object itself is all the scan has to go on. Any alternative that moves the "raw or pointer" information onto the pointer instead of the object breaks that assumption.

3.4.1 Alternatives

One alternative would be to use 2 tag bits on the pointer itself, already encoding what kind of value it points to. This runs into exactly the problem above: the sequential to-space scan never has access to the pointer that led to an object, so the tag would have to be duplicated onto the object anyway, or the sequential scan would have to be replaced with an explicit worklist - giving up the main advantage of the algorithm. It is also harder to extend,

since the number of tag bits needed grows with the number of distinct pointer kinds.

Another alternative would be to use two separate address spaces, one for pointers and one for raw values. This would be extensible and clean but is likely overkill for this thesis.

Because of these reasons, the simplicity of encoding the type in the header - which fits how the collector already traverses memory - was selected.

3.5 Generalizing the Float Box

The float box can be generalized to hold an arbitrary number of raw values instead of exactly one, without any change to the front end or to how tuples work.

I kept the box and the tuple as separate concepts rather than merging them. A tuple containing a float is unchanged: each float is still its own length-one box, referenced by a pointer from the tuple's field. What changed is the box-construction mechanism itself, which now takes a length n - every existing call site still passes $n = 1$.

Nothing yet constructs a box with $n > 1$; this generalizes the mechanism, not any consumer of it. Reading a boxed float always accesses position 0, which is a fixed offset regardless of n, so this required no change.

I did not cap the number of elements a box can hold. The header only spends two of its 64 bits on the forwarding and raw-data flags, leaving roughly 62 bits for the length - far beyond the required 2^{16} or 2^{32}. A cap would add complexity for no benefit, since every value of n is chosen by the compiler itself at compile time.

3.5.1 Alternatives

Representing an "array of floats" as a special-cased tuple - detecting an all-float tuple and storing its elements inline - was rejected, since it conflates two things that are cleanly separate today: a tuple's fields may independently be integers, pointers, or boxed floats, while a box's payload is exclusively one kind of raw data.

A dedicated bit field for the count, separate from the header's existing length field, was also rejected: the length field was never actually limited to one word, only ever used that way. Reusing it keeps the fix backward compatible - for $n = 1$ it produces exactly the same header and collector behaviour as before.

3.5.2 Possible Improvements

An array type built on top of this - fixed, compile-time-constant size and indices - would be the natural next step. A genuinely runtime-sized or runtime-indexed array would additionally require computing addresses from a register, since RISC-V's loads and stores only support a base register plus an immediate offset.

4 Implementation

The integration of floating-point operations requires a full-vertical extension across the compiler pipeline. This chapter details the type systems, lowering mechanisms, and code generation required to support 64-bit double-precision floats within the RV64GC architecture.

4.1 Interpreter Extension and Phase-Driven Design

Before implementing the concrete compiler backend, the language's dynamic semantics were established by extending the reference interpreter. Because the interpreter defines the ground-truth behavioral specification of the language, this execution layer must be finalized first to ensure semantic parity with the compiler's eventual target output.

To establish clear verification metrics, a differential test suite containing half a dozen Python programs was constructed. These test cases duplicated existing integer-based control flow, arithmetic expressions, and native built-in functions (such as `print()`), substituting integer literals with floating-point values. The immediate implementation goal was to allow the interpreter to execute this test suite natively without throwing exceptions, matching Python's runtime behavior exactly.

Extending the interpreter required systematic modifications across three core subsystems of the pipeline: the abstract syntax tree (AST) parser, the static type-checker, and the evaluation semantics engine.

4.1.1 Syntactic Extensions (The Parser)

Extending the frontend parser was highly deterministic because the host language's `ast` module natively recognizes floating-point tokens. Rather than extending the existing literal node, a separate `EConstFloat` node was introduced alongside `EConst`, present in every AST from the source-level representation onward. Additionally, the type-mapping routines were extended to recognize and propagate explicit variable type annotations, allowing patterns such as `a : float = 3.14` to pass successfully through the front-end token stream.

4.1.2 Static Semantics and the 64-Bit Precision Decision

The primary design challenge emerged during the static analysis phase within the type-checking subsystem. Because the compiler targets the RV64GC architecture—which natively supports both single-precision ('F') and double-precision ('D') floating-point extensions—implementing 32-bit floats offered tempting advantages in implementation expediency, specifically minimizing heap pressure and reducing the footprint of boxed values. However, forcing a 32-bit representation would introduce semantic drift from standard Python behavior, which treats all floating-point numbers as 64-bit doubles; common decimals like `0.1` or `3.14` would suffer silent precision loss. This directly conflicts with the design philosophy established in Chapter 3, which prioritises preserving Python's semantics over efficiency and implementation simplicity.

A hybrid architecture—selectively flagging exact base-2 powers (e.g., $0.5 = 2^{-1}$) as 32-bit floats and remaining decimals as 64-bit doubles—was also explored and rejected for the same reason: Python does not expose mixed-precision literals, and forcing such a distinction would create an un-Pythonic dialect while introducing significant complexity into the code generator.

To preserve strict semantic fidelity, I opted for a unified 64-bit double-precision model targeting the RISC-V 'D' extension, accepting the trade-off

of higher memory overhead. Consequently, the type validator enforces a single constraint: verifying that literals do not exceed the 64-bit IEEE 754 infinity threshold.

4.1.3 Type-System Generalization

Beyond validating literals, the type-checker's expression evaluation routines needed a clean refactoring. Originally, the compiler assumed almost everything was an integer, evaluating binary operators and functions like `print()` by checking them directly against a hardcoded `TInt()` type. To support floating-point numbers without copying and pasting the same validation logic everywhere, this rigid approach was replaced with a more flexible system. A new helper function, `check_types_supported`, was introduced to validate expressions against a list of acceptable types instead of just one. For this first iteration, operands must still match exactly—meaning mixed-type operations like `float + int` are blocked for now; this constraint is relaxed later, in Section 4.6. However, this refactor ensures that expressions evaluate to and return their correct type primitive (like `TFloat()`) rather than defaulting to an integer, laying the clean groundwork for future cross-type arithmetic.

4.1.4 Interpreter Semantics

Extending the runtime behavior in the interpreter was the most straightforward step because the interpreter itself is written in Python, which already supports 64-bit floating-point math natively.

In `semantics.py`, the evaluation rules for binary expressions were simply updated to accept float values alongside integers. When the interpreter encounters an operation like addition, it handles the AST nodes by passing the raw values directly to Python's native operators. By leveraging the host language's execution environment this way, the interpreter was immediately able to run the floating-point test suite with perfect accuracy.

4.2 Heap Allocation for Floats

The decision to box every float carried two practical consequences for the allocation pass. From this pass onwards, a float is represented as a pointer to its box, so any operation that consumes a float must first unbox it to recover the underlying value, and any operation that produces a float must box the result again before it can be stored back on the heap. The difficulty is that the allocation pass must know, for each operand, whether it is a float; otherwise it would apply the arithmetic to the box pointer rather than to the value it points to. The type information needed to make this decision was computed earlier, by the type checker, so the task was to make that information available at this point in the pipeline and to consult it for each operand.

The first step was a function that determines whether a given expression yields a float. Whenever it does, the uniform boxing design dictates that the operand must be unboxed before the operation and, where the operation itself yields a float, boxed again afterwards.

A complication arose for values held inside tuples (and for closures, functions, and objects, which are all represented as tuples at this stage). The type checker does verify the type of every element of a tuple, but that per-element information was not preserved beyond type checking, so by the time the allocation pass ran there was no way to recover whether the element at a given position was a float. The expression's shape alone is sufficient for literals and for nested arithmetic, but not for accesses into a tuple or for plain variables, where the type is not visible in the expression itself. To resolve this, I returned to the type checker and preserved the type information it had already derived.

4.2.1 A Dictionary to Preserve Type Information

The result was a dictionary that, for each such structure, uses its identifier as the key and stores its fields or elements together with their corresponding types as the value. This dictionary is returned from the type checker to the main compiler driver, stored there, and passed on to every pass that requires it.

Up to this point in the pipeline, four passes consume the dictionary. The first is uniquify, which renames variables and must therefore update the dictionary's keys so that the recorded types remain reachable under the new names. The second is closure conversion, discussed below. The third is monadic normalisation, discussed in Section 4.4.1, which must propagate types onto the fresh temporaries it introduces. The fourth is the heap allocator, for which the dictionary was originally introduced, where it is used to decide whether a given element is a float.

4.2.2 Preserving Types Through Closure Conversion

Closure conversion runs between uniquify and the heap allocator, and unlike uniquify it does not merely rename variables - it moves a lambda's free variables and parameters into an entirely new top-level function, generated on the fly under a fresh label. Any type information recorded under the enclosing function's identifier is therefore no longer reachable from within the new function's scope, precisely where the allocator would later need it to decide whether a captured variable or parameter is a float.

To keep this information reachable, closure conversion builds a small local type map for each lambda, restricted to its free variables and parameters, and registers it in the dictionary under the new function's own label, alongside the maps for the functions already present in the source program. This way, every function the allocator later encounters - whether written directly by the programmer or generated during closure conversion - has its own entry

in the dictionary, keyed consistently by function identity.

4.2.3 Handling Floats inside the Heap Allocator

When one or more operands of an operation are floats, the allocator emits a block that first extracts each float value from its box, then performs the operation on the unboxed values, and finally, where the result is itself a float, stores it back into a freshly allocated box. When none of the operands are floats, the operation is emitted unchanged, exactly as before.

4.3 Float Literal Support End to End

With the heap-based boxing design fully implemented, the next task was to support float literals through the entire compiler pipeline, from instruction selection down to executable assembly. The test program `float_literal.py` served as the target: it assigns a float literal to a variable and prints it.

4.3.1 Instruction Selection

The first pass that required modifications was instruction selection. As a prerequisite, I extended the register library to include all 32 floating-point registers defined by RISC-V: the temporary registers `ft0`–`ft11`, the saved registers `fs0`–`fs11`, and the argument registers `fa0`–`fa7`. Under the boxed-float strategy most of these registers are not yet used, since every float value resides on the heap and is only loaded into a register for a single operation before being stored back. Nonetheless, defining the full register set lays the groundwork for future optimisations such as keeping floats in registers across operations or spilling to the stack instead of the heap.

Inside the pass itself, I threaded the types dictionary from the compiler driver down to instruction selection so that the pass could determine whether a value should be directed to a floating-point register or an integer register.

Accessing a boxed float requires unboxing: loading from byte offset 7 of the tagged heap pointer to reach the payload field of the box.

4.3.2 Patch Instructions for Literals

The final pass that required changes was patch instructions, which lowers abstract move operations into concrete RISC-V assembly. To support this, I first extended the RISC-V AST with a new instruction node for floating-point moves. RISC-V provides three variants: `fmv.d.x` (integer register to float register), `fmv.x.d` (float register to integer register), and `fmv.d` (float register to float register). I defined all three in the AST.

Inside the pass, when a move instruction is encountered, I introduced a four-way case distinction based on whether the source and destination are floating-point or integer registers. Each combination selects the appropriate move instruction: `fmv.d` when both are float registers, `fmv.d.x` when only the destination is a float register, `fmv.x.d` when only the source is a float register, and the standard integer `add` (used as a move) when neither is.

4.3.3 Extension of the Runtime

To support printing floats, I added a `print_float` function to the C runtime (`runtime.c`). This function receives a double-precision value in `fa0`—following the RISC-V calling convention for floating-point arguments—and prints it using `printf`. This mirrors the existing `print_int64` function that the base compiler uses for integers.

4.3.4 Walkthrough of the Generated Assembly

To illustrate the end-to-end mechanics, consider how the compiler handles the statement `a: float = 3.14`. The float value is converted to its IEEE 754 bit representation as a 64-bit integer and loaded into an integer register using

`li`. It is then stored at byte offset 7 from the tagged heap pointer, which places it in the payload field of the box:

```
li      a2, 4614253070214989087
add     t0, zero, a2
sd      t0, 7(a1)
```

When the program calls `print(a)`, the compiler emits instructions to unbox the float: it loads the raw bits from the heap back into an integer register, then uses `fmv.d.x` to reinterpret those bits as a double-precision value in the floating-point register `fa0`, and finally calls the runtime's `print_float`:

```
ld      a1, 7(a1)
fmv.d.x fa0, a1
call    print_float
```

4.4 Arithmetic for Floating-Point Values

With literals and printing working, the next step was arithmetic. The base compiler only supported addition and subtraction for integers, so multiplication (`*`) and division (`/`) had to be added to the parser, type checker, and every intermediate AST regardless of floats. For floating-point values, RISC-V provides native double-precision instructions for all four operations: `fadd.d`, `fsub.d`, `fmul.d`, and `fdiv.d`.

4.4.1 Transient Use of Float Registers

The central design choice for arithmetic was to keep float registers as a transient detour rather than a place where values live. The instruction selection pass emits the following sequence for every float operation: move both unboxed operands from integer registers into `fa0` and `fa1` with `fmv.d.x`, perform the operation (e.g. `fadd.d fa0, fa0, fa1`), move the result back to an integer register with `fmv.x.d`, and box it on the heap immediately.

Float registers are therefore never live across instruction boundaries from the register allocator's perspective.

This approach required two supporting changes. First, the heap allocation pass already introduces temporary variables for each intermediate result of a nested expression. These temporaries must be recorded in the type dictionary as `TFloat` so that downstream passes can identify them. Second, the instruction selection pass must distinguish float operations from integer ones by checking the operand types. However, the monadic normalisation pass—which flattens nested expressions into a sequence of assignments to fresh temporaries—originally did not have access to the type dictionary, leaving its temporaries untyped. To solve this, I threaded the type dictionary through the monadic pass. When the pass creates a new temporary for a non-atomic expression such as `ETupleAccess(EVar(v), 0)` where `v` is typed as `TFloat`, the temporary inherits that type. This ensures that operand types are available in instruction selection uniformly for both arithmetic and comparisons.

4.4.2 Registering Float Registers in the Allocator

Because `fa0` and `fa1` now appear explicitly in the instruction stream, the register allocator must know about them: any register that shows up in the interference graph needs a colour mapping, otherwise the graph colouring algorithm crashes when it encounters an unknown register as a neighbour of a variable. I added both registers to the blacklist at the beginning of `REG_ORDER`, which assigns them a negative colour. This ensures the allocator recognises them but never assigns a variable to them—which is exactly the desired behaviour, since float values are boxed to the heap immediately and never need to be spilled to a float register.

4.4.3 Walkthrough of the Generated Assembly

The generated code for a float addition such as `a + b` mirrors the unbox–operate–box pattern. Both operands are loaded from the heap, moved into float registers, added, and the result is stored back:

```
ld      a3, 7(a3)
fmv.d.x fa0, a2
add     a2, zero, a3
fmv.d.x fa1, a2
fadd.d  fa0, fa0, fa1
fmv.x.d a2, fa0
add     t0, zero, a2
sd      t0, 7(a1)
```

The first two lines unbox the first operand: `ld` loads the raw bits from the heap (at tagged offset 7), and `fmv.d.x` reinterprets them as a double in `fa0`. Lines three and four do the same for the second operand into `fa1`. The `fadd.d` performs the addition. Finally, `fmv.x.d` moves the result back to an integer register, and `sd` stores it into a freshly allocated box on the heap.

4.5 Comparing Operations for Floats

With arithmetic in place, the next step was comparisons. The RISC-V AST had to be extended with three native double-precision comparison instructions: `feq.d`, `flt.d`, and `fle.d`. As with integers, comparisons appear in two contexts that must be handled differently: assigning the result of a comparison to a variable, and branching on a comparison (e.g. `if`/`else`).

4.5.1 Assigning Comparison Results

When a comparison result is assigned to a variable, the compiler loads both operands, checks their types, and—if they are floats—moves them into the

designated float registers to perform the comparison. The result is a 0 or 1 in an integer register, which is then tagged like any other integer and stored in the destination variable.

RISC-V only provides `feq.d` (equal), `flt.d` (less than), and `fle.d` (less or equal). The missing operators are derived with simple transformations: greater-than and greater-or-equal swap the operands (`a > b` becomes `flt.d rd, fb, fa`), while not-equal uses `feq.d` followed by an `xor` to invert the result.

4.5.2 Branching on Comparison Results

The branching case differs structurally from the integer path. For integers, RISC-V can compare and branch in a single instruction (e.g. `blt rs1, rs2, label`). For floats, this is not possible—no float branch instructions exist. Instead, the compiler emits the float comparison into a temporary integer register, materializes zero into a second register, and branches with `bne` between the two.

4.5.3 Walkthrough of Generated Assembly

The following walkthrough shows the branching case. As with any float operation, both values are unboxed from the heap and moved into float registers. The `flt.d` instruction compares them and writes 1 or 0 to an integer register. The result is then tagged and the branch is taken accordingly:

```
ld      a2, 7(a2)
ld      a1, 7(a1)
fmv.d.x fa0, a2
fmv.d.x fa1, a1
flt.d   a1, fa1, fa0
slli    a1, a1, 1
li      t1, 0
bne     a1, t1, body_6
j       orelse_7
```

4.6 Type Coercion for Mixed Integer and Float Operations

The first change required was relaxing the type checker. Previously, binary operations required both operands to have the same type. This constraint was loosened so that operands need only be individually valid for the operation—for instance, both `TInt` and `TFloat` are permitted for arithmetic—without requiring them to match. When at least one operand is a float, the result type is `TFloat`.

With the type checker permitting mixed-type expressions, the heap allocation pass handles the actual coercion. This pass already inspects each operand to determine whether it is a float and requires unboxing. When one operand is a float and the other is an integer, the pass wraps the integer in the `int_to_float` unary operation introduced in Section 3.3. The wrapped expression flows through the monadic and explicate passes without special handling.

In instruction selection, the `int_to_float` operation is lowered to a short sequence: the integer is loaded into a temporary register, untagged by shifting right by one bit (since the compiler stores integers in tagged form), and then

converted to a double-precision float using the RISC-V `fcvt.d.l` instruction. The resulting float value is moved back to an integer register, ready for use in the subsequent arithmetic operation.

4.6.1 Walkthrough of Generated Assembly

The following listing shows the generated code for `3.2 * 3`, where the first operand is a float and the second is an integer. The float is unboxed from the heap as usual, while the integer literal 3 is loaded in its tagged form (value 6), untagged, and converted to a double before the multiplication:

```
ld      a3, 7(a3)
li      a2, 6
srli    a2, a2, 1
fcvt.d.l fa0, a2
fmv.x.d a2, fa0
fmv.d.x fa0, a3
fmv.d.x fa1, a2
fmul.d  fa0, fa0, fa1
fmv.x.d a2, fa0
```

The first line unboxes the float operand. Lines two through five handle the integer: `li` loads the tagged value 6, `srli` untags it to 3, and `fcvt.d.l` converts the integer 3 to the double 3.0 in `fa0`. The result is then moved back to an integer register with `fmv.x.d`. Finally, both operands are moved into float registers and the multiplication is performed.

4.7 Extension of the Garbage Collector

The Garbage Collector has to be extended to know whether an allocated object is a boxed float or a tuple/closure. Because I decided to encode this directly in the object's header, the implementation only required small, local

changes to the compiler and the runtime, rather than a new address space or a second pointer encoding.

4.7.1 Extending the Compiler

The decision to box a float, and the code that computes and stores its value, is made earlier in the pipeline, in `pass_5_6_alloc.py`: whenever an expression produces a float, that pass allocates a single-slot container for it and emits a statement that writes the computed value into that slot, exactly like writing into any other tuple field.

By the time `pass_8_9_select` sees this allocation, boxing has already been decided - its job is only to choose the correct header encoding. When it lowers the allocation into a concrete header constant, it checks whether the variable being allocated is typed as a float. If it is, it sets bit 1 of the header in addition to the length that is already encoded in the upper bits; if not, bit 1 is left clear. The payload itself does not need any float-specific handling in this pass, since it is written using the same generic tuple-subscript-store logic that handles any other tuple field.

4.7.2 Extending the Runtime

The collector's sweep over to-space normally treats every word following an object's header as a potential pointer and checks it accordingly. This is exactly what would go wrong for a boxed float, since the raw bits of a double can easily satisfy that check by coincidence.

The extended sweep loop first reads bit 1 of the header. If it is set, the loop skips both the header and the single payload word without inspecting the payload at all, advancing its scan pointer by two. If it is not set, the loop proceeds as before, checking each of the object's fields individually. This one bit is enough to keep the payload of a boxed float from ever being misread as a heap pointer during collection.

4.8 Generalizing the Float Box

Prior to this extension, every place that boxed a float - a literal, the result of a unary operation, and the result of a binary operation - duplicated the same steps: check whether a collection is needed, allocate the payload, and register the resulting variable's type. I consolidated this into a single helper, `make_float_box(n, types)`, in `pass_5_6_alloc.py`, which performs the garbage-collection check and the allocation for a box of n raw words and returns both the fresh box identifier and the statements needed to construct it. `EConstFloat`, `EOp1`, and `EOp2` all now call this helper with $n = 1$, rather than repeating the same allocation logic three times.

Instruction selection required no changes. The existing header-construction logic in `pass_8_9_select.py` already computed `(num_elems « 2) | 0b10` generically from whatever length the allocation was given - it had simply never been called with anything other than 1.

The one change genuinely required was in the runtime. The collector's sweep, on encountering a raw object, previously skipped exactly two words unconditionally, regardless of the header's own length field. I changed this to skip `1 + count` words, where `count` is read directly from the header, exactly as it already was for scannable objects. For $n = 1$ this computes $1 + 1 = 2$, identical to the previous behaviour, so no existing test could regress; for any $n > 1$ the collector now correctly treats the box as a single unit and skips the entire payload without scanning it for pointers.

5 Evaluation

Development was based on 20 predesigned tests intended to ensure basic functionality and feature support, along with 3 additional tests written after completing the implementation to stress-test it and cover edge cases, on top of the 141 tests already present from the original compiler.

The practice of testing against the interpreter as the language definition was continued from earlier work: before writing any compiler code, I first defined the expected behavior via the interpreter.

One caveat did arise: the float representation of the print function differs between Python and C. Although the underlying value is the same, the C representation uses more digits after the decimal point (e.g. 3.2 vs. 3.200000000), so outputs were compared for similarity rather than exact equality, to avoid false negatives.

This also surfaced a related, known formatting difference: occasionally the two rendered outputs disagree in their final decimal digit, e.g. 3.100000 versus 3.099999 for what is otherwise the same value. The underlying IEEE 754 binary representation is identical in both languages; the difference comes from how each language's print function rounds that binary value into a decimal string. C's `printf("%f")` always rounds to a fixed six decimal places, while Python's `print` uses the shortest decimal string that round-trips back to the exact same binary value. When the true binary value sits close to a decimal rounding boundary, these two independent rounding strategies can disagree in the last digit, even though neither is incorrect.

5.1 Literals and Arithmetic

These tests cover storing floats in variables, printing them, and performing basic arithmetic (addition, subtraction, and so on). Since this was a required feature, the implementation had to guarantee that it was fully supported.

5.2 Comparison and Coercion

This builds on the basic arithmetic tests. Comparisons required a different set of operations to be supported, and their results had to be handled differently from arithmetic results. Tests such as compare capture this by storing the result of a greater-than-or-equal comparison in a variable.

For coercion, I tested that the result has the correct representation and that operands of mixed types are converted correctly before the operation is applied.

5.3 Control Flow

These tests were targeted at making sure the extension did not break already-working features: if and while statements should not behave any differently with floats than they do with other data types, and these tests confirm that this holds.

5.4 Functions and Closures

These tests were designed to check that function return types were propagated correctly. In doing so, they uncovered a bug in which type information for a closure's captured variables and parameters was lost during closure conversion. As a result, references to captured float variables inside a closure's body were treated as plain integers further down the pipeline.

The fix was to thread the type dictionary through the closure conversion pass and to record a local type map for each generated closure function (see chapter 4 for details).

5.5 Data Structures and Objects

The main purpose of these tests was to exercise the functionality of boxed floats. Since every object, function, and data structure that has a float as a field is itself represented as a tuple, any such float had to be boxed within it.

Because this transformation was already implemented, these tests checked whether anything broke as a result of the new way float values were handled.

They also verify that boxing and unboxing inside these structures is handled correctly. The design of these tests influenced decisions such as boxing itself, since they were written before the corresponding implementation, as a way to explore how the internals should work and what needed to be supported.

In that sense, these tests directly laid the foundation for the decision to box floats.

5.6 Stress tests and Edge Cases

After finishing the core implementation, I wrote three additional test files specifically designed to test edge cases and to stress-test the garbage collector.

Even under roughly 2000 heap allocations performed in a tight loop, the garbage collector handled collection correctly and reasonably quickly. The same held for the edge-case value tests, which covered very small, very large, and signed-zero floats.

The test involving nested, chained closures revealed a real limitation: using the result of one closure call as the argument to another produced an incorrect value. The most likely cause lies somewhere in code generation

or register allocation for chained closure calls, though the exact root cause was not conclusively identified within the scope of this thesis. This test was therefore moved into a separate, clearly labeled limitations folder and is discussed here rather than counted among the passing test suite.

6 Conclusion

The work of this thesis was to extend a Nanopass Python-to-RISC-V compiler, originally developed for the Compiler Construction lecture and supporting bools, ints, and tuples, with support for floating-point values across all operations.

6.1 Deliverables

The ASTs were extended with a new `ConstFloat` type, and instruction selection was extended to emit RISC-V floating-point instructions for all supported operations. A new unary operation was introduced to convert an int to a float where necessary, for coercion.

To keep track of the types determined by the type checker, a type dictionary was introduced and threaded through the pipeline. Where later passes performed renaming or restructuring, such as during closure conversion, the dictionary was updated accordingly so that the recorded types remained valid.

Floats leaving a register were always boxed and stored on the heap, giving a single, uniform representation to work with at every later stage. The garbage collector was extended to support this representation correctly.

A set of 23 tests was written to validate expected behavior across all features. 22 of these are part of the passing test suite; the remaining one was isolated as a documented limitation and is discussed below.

6.1.1 Limitations

Always boxing floats is memory-inefficient and leads to more frequent garbage collector activity.

There is a known bug in how chained, nested closures are compiled, most likely located in code generation or register allocation (see chapter 5).

While reviewing the runtime's register classification, I also found that the check identifying floating-point registers matched any register name starting with the letter `f`, which would have misclassified the frame pointer (`fp`) as a floating-point register. This was never triggered in practice, since `fp` never reaches that dispatch as a move operand, but the check has since been tightened to match the exact float-register naming convention instead.

6.2 Future Work

A possible optimization for the compiler would be to allow floats to be spilled to the stack as well. The main obstacle is the bit-tagging scheme used to distinguish ints and pointers on the stack (see chapter 3).

One way to achieve this would be a dual stack, with one designated region for floats and one for ints. Another would be to extend the type-tracking mechanism to inform the register allocator directly, allowing floats and ints to share the same stack space while remaining distinguishable.

Bibliography

C. J. Cheney. A nonrecursive list compacting algorithm. *Communications of the ACM*, 13(11):677–678, 1970.

IEEE. IEEE standard for floating-point arithmetic, 2019. IEEE Std 754-2019 (Revision of IEEE Std 754-2008).

Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008. ISBN 9780132350884.

Python Software Foundation. ast — abstract syntax trees. `https://docs.python.org/3/library/ast.html`, 2026a. Python 3 documentation, accessed 2026.

Python Software Foundation. Floating-point arithmetic: Issues and limitations. `https://docs.python.org/3/tutorial/floatingpoint.html`, 2026b. Python 3 documentation, accessed 2026.

Jeremy G. Siek. *Essentials of Compilation: An Incremental Approach in Python*. MIT Press, 2023. ISBN 9780262048248.

Andrew Waterman, Yunsup Lee, David A. Patterson, and Krste Asanović. The risc-v instruction set manual, volume i: User-level isa, version 2.1. Technical Report UCB/EECS-2016-118, EECS Department, University of California, Berkeley, 2016.